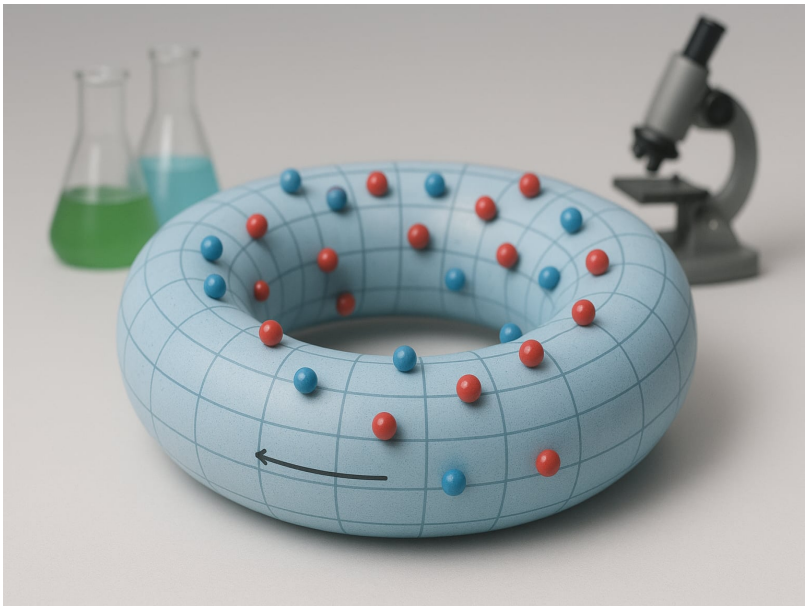
Bakterie (bakterie-grid)

Limit pamięci: 256 MB Limit czasu: 1.00 s

To zadanie jest interaktywne. Po wypisaniu każdego wiersza należy opróżnić bufor. W C++ możesz użyć **cout « flush**, w Pythonie – **sys.stdout.flush()**. Należy ściśle trzymać się protokołu opisanego w sekcji *Interakcja* – niewczytanie wiadomości wysłanej przez interaktor skutkuje werdyktem *wrong answer* lub innym.

Na planszy $N \times M$ zalęgła się pewna niezerowa liczba bakterii. Plansza ta ma dosyć osobliwe własności. Jej wymiary N i M to dwie różne liczby pierwsze. Ponadto, plansza ma kształt torusa, czyli pola w górnym i dolnym rzędzie sąsiadują ze sobą nawzajem, podobnie jak pola w pierwszej i ostatniej kolumnie. Dokładniej, jeśli weźmiemy pole (X, Y), dla $1 \leq X \leq N$, $1 \leq Y \leq M$, to jego dolnym sąsiadem jest pole $(X + 1, Y)$ dla $X < N$ i $(1, Y)$ dla $X = N$, natomiast prawym sąsiadem jest $(X, Y + 1)$, gdy $Y < M$ i $(X, 1)$ dla $Y = M$. Bakterie dzielą się na dwa typy, `B` i `K`. Bakterie typu `B` w każdej minucie przesuwają się w dół do sąsiedniego pola, natomiast te typu `K` przesuwają się w prawo do sąsiedniego pola. Wiadomo, że w każdym polu jest co najwyżej jedna bakteria danego typu[1]. Dodatkowo, początkowo co najmniej jedno pole planszy jest puste[2].



Model atomu Thomsona. Cambridge, 1904.

Obserwacja stanu planszy pozwala nam poznać, w których komórkach są bakterie, nie pozwala jednak dowiedzieć się, jakich są typów. Należy zebrać wszystkie bakterie z planszy wykonując pewien ciąg operacji, każda jednego z dwóch typów:

1. Zbadanie bakterii z pola (X, Y). Nie można tego wykonać, jeśli pole jest puste. Po operacji wszystkie bakterie z tego pola zostają zabrane z planszy, a następnie są poddawane badaniom. Dzięki badaniom można ustalić czy w komórce znajdowała się tylko jedna bakteria, czy też dwie różnych typów. Zbadanych bakterii nie odkładamy już na planszę. Ze względów praktycznych nazwiemy badanie *jakościowym*, jeśli bakteria była tylko jedna.
2. Czekanie przez T minut. Po tym czasie wszystkie bakterie przesuną się o T pól w dół lub prawo, zależnie od tego, jakiego są typu. Po tym czasie będziemy mogli kolejny raz obejrzeć stan planszy i dowiedzieć się, na których polach stoją bakterie. Warto zauważyć, że czekanie to następuje po poprzednim, zatem nie jest to stan planszy po czasie T od początku, tylko stan po czasie T od poprzedniego czekania.

Oprócz pozbycia się wszystkich bakterii z planszy chcemy to zrobić w taki sposób, aby jak najwięcej wyciągnąć z eksperymentu. Dokładniej, chcemy optymalizować dwa parametry:

[1] Łatwo zauważyć, że nie zmienia się to po ruchach bakterii.

[2] Ale może się okazać, że po pewnym czasie wszystkie pola będą zajęte.

- jakościowość badań, czyli to jaka część wszystkich bakterii z planszy została ostatecznie zbadana w sposób jakościowy
- wydajność badań, czyli to ile razy będziemy musieli wykonać obserwację planszy po czekaniu

Twoim zadaniem jest wykonać ciąg operacji, który jest możliwie udany, pod kątem powyższych czynników. Szczegóły znajdują się w sekcji punktacja. Aby otrzymać maksymalną liczbę punktów, należy czekać co najwyżej 9 razy i wykonać wyłącznie jakościowe badania.

Interakcja

Interaktorka nie jest adaptacyjna - początkowe pozycje i typy bakterii są ustalone i nie zależą od wykonywanych operacji.

Na początku należy wczytać liczby całkowite N, M, S, oznaczające odpowiednio liczbę wierszy i kolumn planszy, oraz numer podzadania. W kolejnych N wierszach jest opisany początkowy stan planszy. W każdym wierszu jest napis złożony z M znaków `0` i `1`, oznaczających, że pole jest puste lub zajęte, odpowiednio.

Następnie należy wykonać pewną liczbę operacji.

Operację pobrania bakterii z danego pola należy wyrazić poprzez `BADAJ X Y`, gdzie $1 \leq X \leq N, 1 \leq Y \leq M$ oznaczają numer badanego pola. Nie wolno wykonać tej operacji, jeśli pole (X, Y) jest puste. W odpowiedzi należy wczytać jedną liczbę, oznaczającą liczbę pobranych bakterii. Jeżeli po tej operacji plansza będzie pusta, należy (po wczytaniu odpowiedzi) zakończyć działanie programu. Zwróć uwagę, że nie jest wypisywany stan planszy, zatem należy samemu (bazując na wyniku ostatniego wywołania `CZEKAJ` lub, jeśli takowego nie było, początkowym stanie) kontrolować jej niepustość po każdej operacji.

Operacja czekania jest reprezentowana jako `CZEKAJ T`, gdzie $1 \leq T \leq 1000$ jest czasem czekania. W odpowiedzi należy wczytać N wierszy, opisujących stan planszy po odczekaniu tego czasu (czyli stan po tym jak każda bakteria przesunęła się o T).

Ograniczenia

$2 \leq N, M \leq 23$, $N \neq M$ są pierwsze.

W każdym polu jest co najwyżej jedna bakteria każdego typu. Początkowo co najmniej jedno pole planszy jest puste, ale nie wszystkie.

Punktacja

Liczba punktów **zależy od dwóch poniższych parametrów**:

- współczynnik za jakościowość liczony jest jako $(\frac{A}{B})^3$, gdzie A jest łączną liczbą jakościowych badań a B jest łączną liczbą bakterii na początku
- współczynnik za wydajność, liczony według poniższej tabelki:

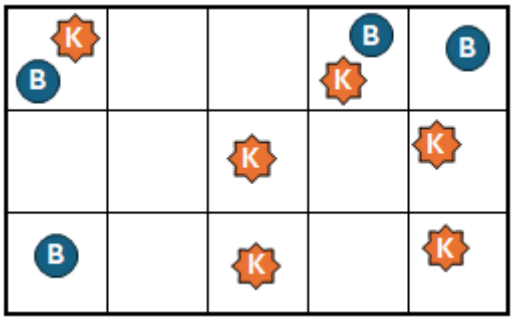
Liczba Q wywołań funkcji `CZEKAJ`	**Współczynnik wydajności w procentach**
$Q \leq 9$	100
$9 < Q \leq 1000$	$100 - 5 \cdot \log_2(Q - 8)$
$1000 < Q$	0

Procent liczby punktów otrzymanych za dany test to iloczyn dwóch powyższych współczynników.

Mamy następujące podzadania:

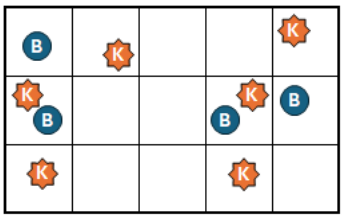
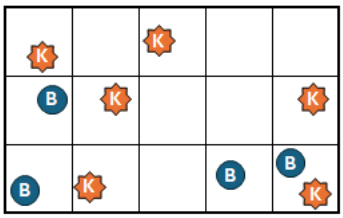
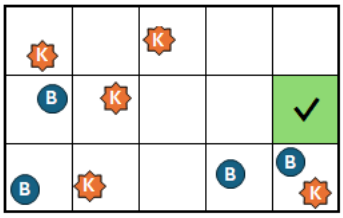
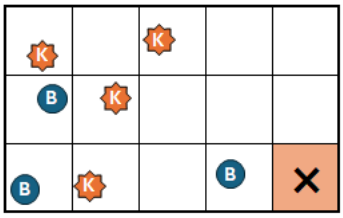
S	**Ograniczenia**	**Punkty**
1	Test jest taki jak na rysunku i w przykładowej interakcji	8
2	Pewien wiersz i pewna kolumna są początkowo puste	17
3	Pewna kolumna jest pusta	19
4	Po upływie pewnej liczby minut każde pole zawiera co najwyżej jedną bakterię	30
5	Brak ograniczeń	26

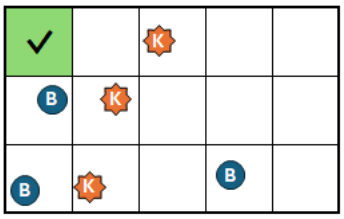
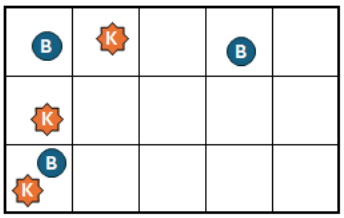
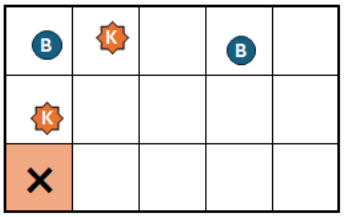
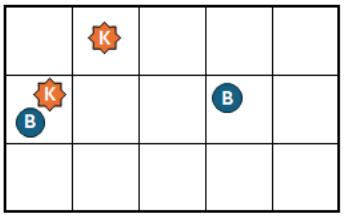
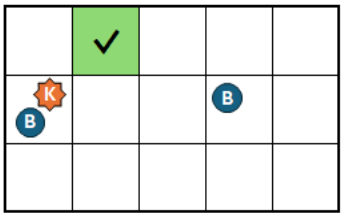
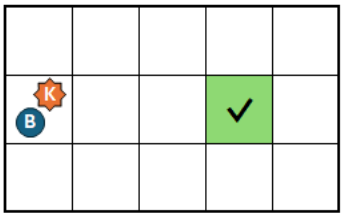
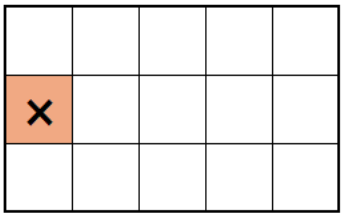
Ilustracja do opisu podzadania 1:



Pozycje bakterii są ustalone na początku i nie zależą od wykonywanych operacji.

Przykładowa interakcja

Wejście	Wyjście	Rysunki obrazujące stan planszy po kolejnych operacjach
3 5 1 10011 00101 10101		
	CZEKAJ 1	
11001 10011 10010		
	CZEKAJ 1	
10100 11001 11011		
	BADAJ 2 5	
1		
	BADAJ 3 5	
2		

Wejście	Wyjście	Rysunki obrazujące stan planszy po kolejnych operacjach
	BADAJ 1 1	
1		
	CZEKAJ 4	
11010 10000 10000		
	BADAJ 3 1	
2		
	CZEKAJ 1000	
01000 10010 00000		
	BADAJ 1 2	
1		
	BADAJ 2 4	
1		
	BADAJ 2 1	
2		

Odpowiada ona **testowi z podzadania 1**. Współczynnik jakościowości wynosi $(\frac{4}{10})^3 = 6,4\%$ (4 z 10 bakterii zostało zebranych jakościowo), natomiast wydajności 100% (czekano 4 razy, czyli nie więcej niż 9). Zatem program otrzymał $6,4\%$ punktów za test.

Narzędzia do testowania lokalnego i testy przykładowe

W zadaniu jest 5 testów przykładowych, każdy z nich reprezentuje inne podzadanie. Zostaną również udostępniony program **interaktor.cpp** oraz przykładowe błędne rozwiązanie o poprawnym schemacie komunikacji **abc.cpp**. Żeby uruchomić rozwiązanie na teście, należy skompilować rozwiązanie i interaktorkę, a następnie użyć komendy:

`[plik wykonywalny z interaktor.cpp] [test] [plik wykonywalny z rozwiązaniem]`

Na przykład: `./interaktor 0a.in ./abc`

Kiełbasa (kielbasa)

Limit pamięci: 64 MB Limit czasu: 1.00 s

Karol i Andrzej kupili długi sznur kiełbasy. Teraz chcą go podzielić. W każdym ruchu jeden z nich może odciąć najbardziej lewą kiełbaskę, najbardziej prawą albo obie.

Jak wiadomo, zaczyna Karol, a potem ruchy są wykonywane naprzemiennie. Oczywiście, obaj chcą zmaksymalizować uzyskaną długość kiełbasy. Jaka jest największa długość kiełbasy, jaką każdy z nich może uzyskać przy optymalnej grze obu stron?

Napisz program, który wczyta długości kolejnych kiełbasek w sznurze, wyznaczy największe sumaryczne długości, które mogą uzyskać Karol i Andrzej w sposób opisany powyżej, i wypisze je na standardowe wyjście.

Wejście

W pierwszym wierszu wejścia znajduje się dodatnia liczba całkowita N, oznaczająca liczbę kiełbasek w sznurze. W drugim wierszu wejścia znajduje się N dodatnich liczb całkowitych $A_1, A_2, ..., A_N$ oddzielonych pojedynczymi odstępami, oznaczających długości kolejnych kiełbasek w sznurze.

Wyjście

W pierwszym wierszu wyjścia należy wypisać dwie liczby oddzielone pojedynczym odstępem – największą sumaryczną długość kiełbasy Karola i największą sumaryczną długość kiełbasy Andrzeja przy optymalnej grze obu stron.

Ograniczenia

$1 \leq N, A_i \leq 1\,000\,000.$

Częściowa ocena

Podzadanie	Punkty	Warunki
1	15	N $\leq$ 10
2	25	N $\leq$ 1000
3	20	N jest liczbą nieparzystą
4	40	Brak dodatkowych ograniczeń

Przykład

Wejście

```
4
7 6 3 9
```

Wyjście

```
16 9
```

Wyjaśnienie

Karol może na początku odciąć skrajne kiełbaski i uzyskać sumaryczną długość 16. Andrzej dostanie wtedy środkowe kiełbaski o sumarycznej długości 9.

Roboty (roboty)

Limit pamięci: 1024 MB Limit czasu: 1.50 s

Kraina Solve4 jest przedstawiona jako siatka o rozmiarze $(h+2) \times (w+2)$. Wiersze tej siatki są ponumerowane od 0 do $h+1$ z góry na dół, a kolumny od 0 do $w+1$ z lewej na prawo. Komórkę w wierszu r i kolumnie c będziemy oznaczać jako (r, c).

Początkowo wszystkie komórki są białe, z wyjątkiem komórek w **prawej skrajnej kolumnie** (czyli w kolumnie $w+1$), które są czarne.

W każdej z kolumn od 1 do w znajduje się dokładnie **jedna wyróżniona komórka**. Każdą z tych wyróżnionych komórek można pokolorować na czerwono albo na niebiesko. Na brzegu siatki (czyli w skrajnie górnym wierszu, skrajnie dolnym wierszu, skrajnie lewej kolumnie oraz skrajnie prawej kolumnie) nigdy nie ma wyróżnionych komórek.

Na niektórych komórkach w lewej skrajnej kolumnie (kolumna 0) zostaną ustawione roboty. Każdy robot porusza się zgodnie z kolorem komórki, na której aktualnie stoi:

- jeśli komórka jest **biała**, robot idzie o jedną komórkę w prawo,
- jeśli komórka jest **czerwona**, robot idzie o jedną komórkę w górę,
- jeśli komórka jest **niebieska**, robot idzie o jedną komórkę w dół,
- jeśli komórka jest **czarna**, robot **przestaje się poruszać**.

Roboty nie zderzają się ze sobą; każdy porusza się niezależnie od innych. Wiele robotów może stać na tej samej komórce jednocześnie.

Jedno zapytanie opisane jest przez dwie liczby całkowite a i b ($1 \leq a < b \leq h$). Dla danego zapytania, dla każdego c spełniającego $a \leq c \leq b$, na komórce $(c, 0)$ ustawiamy jednego robota. Następnie roboty poruszają się zgodnie z zasadami opisanymi wyżej, aż wszystkie się zatrzymają (na czarnych komórkach w prawej kolumnie).

Dla każdego zapytania wolno nam **dowolnie** pokolorować wszystkie wyróżnione komórki (osobno dla każdego zapytania), wybierając dla każdej z nich kolor czerwony albo niebieski. Naszym celem jest tak dobrać te kolory, aby **zminimalizować liczbę różnych (rozłącznych) komórek**, na których ostatecznie znajdą się roboty po zatrzymaniu.

Dla każdego zapytania należy wyznaczyć tę minimalną możliwą liczbę różnych końcowych komórek, które mogą zostać zajęte przez roboty, przy optymalnym wyborze kolorów wyróżnionych komórek.

Wejście

W pierwszym wierszu wejścia znajdują się dwie liczby całkowite h oraz w, oddzielone pojedynczym odstępem i oznaczające odpowiednio:

- h – liczbę wierszy „wewnętrznych” (bez obramowania),
- w – liczbę kolumn „wewnętrznych” (bez obramowania).

W drugim wierszu znajduje się w liczb całkowitych $x_1, x_2, \ldots, x_w$, oddzielonych pojedynczymi odstępami. Liczby te opisują położenie wyróżnionych komórek:

- w kolumnie 1 wyróżniona jest komórka $(x_1, 1)$,
- w kolumnie 2 wyróżniona jest komórka $(x_2, 2)$,
- $\ldots$
- w kolumnie w wyróżniona jest komórka (x_w, w).

W trzecim wierszu znajduje się jedna liczba całkowita q – liczba zapytań.

W kolejnych q wierszach opisane są zapytania.

W i-tym z tych wierszy znajdują się dwie liczby całkowite a_i oraz b_i ($1 \leq a_i < b_i \leq h$), oddzielone pojedynczym odstępem.

Dla danego zapytania ustawiamy po jednym robocie na komórkach $(c, 0)$ dla wszystkich c spełniających $a_i \leq c \leq b_i$.

Dla **każdego** zapytania możemy niezależnie dobrać kolory (czerwony/niebieski) wszystkich wyróżnionych komórek.

Wyjście

Twój program powinien wypisać q wierszy.
W i-tym wierszu powinna znaleźć się jedna liczba całkowita – minimalna możliwa liczba **różnych** komórek, na których mogą ostatecznie zatrzymać się roboty w i-tym zapytaniu, przy optymalnym wyborze kolorów wyróżnionych komórek.

Ograniczenia

Dla wszystkich testów zachodzą następujące warunki:

- $1 \leq w, q \leq 200\,000$,
- $2 \leq h \leq 200\,000$,
- $1 \leq x_j \leq h$ dla wszystkich $1 \leq j \leq w$,
- $1 \leq a_i < b_i \leq h$ dla wszystkich $1 \leq i \leq q$.

Podzadania

Podzadanie	Warunki	Punkty
1	$h, w \leq 16$, $q \leq 20$	10
2	$a_i + 1 = b_i$	4
3	$x_1 < x_2 < \cdots < x_w$	12
4	$h, w, q \leq 5\,000$	43
5	brak dodatkowych ograniczeń	31

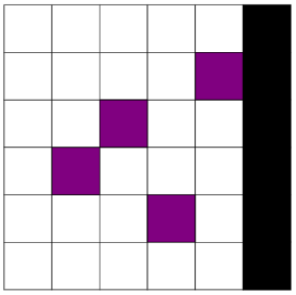
Przykład

Wejście	Wyjście	Wyjaśnienie

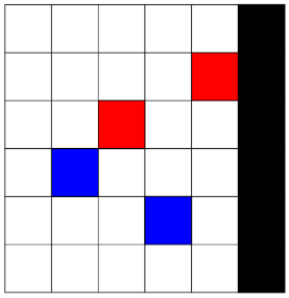
```
4 4
3 2 4 1
3
1 4
1 3
2 4
```

```
2
1
1
```

Plansza jest pokolorowana w taki sposób, z specjalnymi polami zaznaczonymi na fioletowo.



Dla pierwszego zapytania możemy pokolorować wyróżnione komórki następująco:



komórki $(3, 1)$ oraz $(4, 3)$ na niebiesko, a komórki $(2, 2)$ oraz $(1, 4)$ na czerwono, aby otrzymać następujący przebieg ruchów:

- Robot startujący z komórki $(1, 0)$ przechodzi kolejno przez komórki $(1, 1)$, $(1, 2)$, $(1, 3)$, $(1, 4)$, $(0, 4)$, $(0, 5)$ i ostatecznie kończy w $(0, 5)$.
- Robot startujący z komórki $(2, 0)$ przechodzi kolejno przez komórki $(2, 1)$, $(2, 2)$, $(1, 2)$, $(1, 3)$, $(1, 4)$, $(0, 4)$, $(0, 5)$ i ostatecznie kończy w $(0, 5)$.
- Robot startujący z komórki $(3, 0)$ przechodzi kolejno przez komórki $(3, 1)$, $(4, 1)$, $(4, 2)$, $(4, 3)$, $(5, 3)$, $(5, 4)$, $(5, 5)$ i ostatecznie kończy w $(5, 5)$.
- Robot startujący z komórki $(4, 0)$ przechodzi kolejno przez komórki $(4, 1)$, $(4, 2)$, $(4, 3)$, $(5, 3)$, $(5, 4)$, $(5, 5)$ i ostatecznie kończy w $(5, 5)$.

Roboty kończą ruch w 2 różnych komórkach: $(0, 5)$ oraz $(5, 5)$, więc poprawną odpowiedzią jest 2.
Dla drugiego zapytania możemy tak dobrać kolory wyróżnionych komórek, aby roboty startujące z komórek $(1, 0)$, $(2, 0)$ oraz $(3, 0)$ wszystkie kończyły ruch w komórce $(0, 5)$.

Wejście	Wyjście
15 16	2
5 15 3 4 13 8 3 7 14 6 2 10 2 11 12 9 1	
20	3
4 10	2
7 15	2
5 15	3
2 6	1
7 15	1
5 15	3
12 15	1
13 14	3
5 14	2
13 14	1
2 11	1
3 11	3
2 5	3
9 11	4
3 15	1
5 14	2
1 13	1
3 7	
7 12	
4 8	